

Rag at scale

This project is built around a large corpus and addresses the engineering challenges that only appear at scale: indexing pipelines, latency budgets, caching, cost control, and approximate search. This specification outlines what industry RAG actually looks like, separating a robust production system from a standard portfolio demo.

The system in one paragraph

A public search-and-answer service over ~1.5 million text chunks (arXiv abstracts or Wikipedia AI/ML articles). Users ask a question; the system retrieves with hybrid search, reranks, and generates a grounded answer with citations. Alongside the answer it exposes what production teams actually care about: which chunks were retrieved and why, p50/p95 latency, cache hit rate, tokens and cost per query, and live quality metrics. Users can switch retrieval strategies and watch all of it change.

The industry problems you will actually solve

1. Ingestion at scale

Embedding 1.5M chunks is not a for-loop. You need batching, GPU/CPU utilisation, checkpointing so a crash doesn't cost six hours, and parallel workers.

Skills: batch processing, async workers, checkpointing, idempotent pipelines, progress tracking

2. Approximate nearest-neighbour search

Exact search over 1.5M vectors is too slow. You need HNSW or IVFFlat indexes and you need to understand the recall/latency/memory tradeoff and be able to publish the curve.

Skills: pgvector, HNSW & IVFFlat, ef_search/m tuning, recall@k measurement, index build time vs query time

3. Chunking, measured rather than guessed

Fixed-size, recursive, semantic, and structure-aware chunking, benchmarked against each other on the same question set.

Skills: chunking strategies, overlap tuning, benchmark design

4. Hybrid retrieval and reranking

BM25 for exact terms, dense for meaning, Reciprocal Rank Fusion to combine, cross-encoder to rerank the top 50 down to the top 5.

Skills: BM25, dense retrieval, RRF, cross-encoder reranking, two-stage retrieval

5. Caching, at three levels

This is where real money is saved.

- **Embedding cache** - never embed the same query twice
- **Semantic cache** - if a new question is >0.95 similar to a previous one, return the cached answer without calling the LLM at all
- **Response cache** - exact-match on (query, settings)

Skills: Redis, TTL strategy, semantic caching, cache invalidation, measuring hit rate

6. Cost and latency engineering

Publish tokens per query, cost per query, p50/p95/p99 latency, and cache hit rate on the page.

Skills: token accounting, prompt compression, context budgeting, latency profiling

7. Evaluation as a pipeline, not a one-off

A 200-question golden set, RAGAS run automatically in CI, and a regression check that fails the build if retrieval quality drops.

Skills: RAGAS, golden datasets, evaluation in CI (very few candidates have this)

8. Serving under public traffic

Rate limiting, queueing, graceful degradation when the LLM API is slow, structured logging, tracing.

Skills: Redis rate limiting, backpressure, circuit breakers, langsmith structured logs

Architecture

Next.js frontend → SSE streaming → FastAPI

Redis (rate limit, embedding cache, semantic cache)

- BM25 index → metadata filters
- pgvector HNSW → RRF fusion → cross-encoder rerank → LLM

langsmith (traces, cost, latency)

Offline: ingestion workers → chunk → embed (batched) → pgvector + BM25 index

Stack

LAYER	CHOICE
Frontend	Next.js 14, TypeScript, Tailwind
API	FastAPI, async, SSE streaming
Vector store	PostgreSQL + pgvector (HNSW)
Keyword	BM25 (rank_bm25, or Postgres full-text)
Cache/limits	Redis
Embeddings	bge-small-en-v1.5 (CPU, free)
Reranker	bge-reranker-base cross-encoder
LLM	Gemini Flash / GPT-4o-mini
Evaluation	RAGAS
Tracing	langsmith (free tier)
Orchestration	Docker Compose
CI/CD	GitHub Actions

Hosting	Hetzner CX32 (~€12/mo, 8 GB RAM)
---------	----------------------------------

Five weeks

Week 1 - Foundation + deploy

Docker Compose (FastAPI + Postgres/pgvector + Redis) → ingest 10k chunks end to end → dense retrieval working → minimal UI → live on a public URL by day 5

Week 2 - Scale the pipeline

Ingest the full 1.5M chunks: batching, checkpointing, parallel workers → build HNSW index → measure and publish index build time, memory, query latency → tune ef_search, plot the recall/latency curve

Week 3 - Retrieval quality

BM25 → RRF hybrid → cross-encoder reranking → benchmark 4 chunking strategies → build the 200-question golden set → run RAGAS across every configuration → produce the results table

Week 4 - Cost and serving

Three-layer caching → token and cost accounting → rate limiting → SSE streaming → langsmith tracing latency dashboard on the page

Week 5 - Ship it properly

pytest suite → GitHub Actions running lint/test/build/deploy → RAGAS regression gate in CI → README with architecture diagram and every metric → demo GIF → blog post → LinkedIn post

What you publish

- A live URL serving semantic search over 1.5M chunks
- A metrics table: recall@k, p50/p95 latency, cache hit rate, cost/query, RAGAS scores per configuration
- A repo with CI badge, tests, Docker, and a real README
- A blog post on what actually moved retrieval quality

The CV lines this produces

Rag at scale - Production retrieval platform | live URL

- Serves hybrid semantic search over 1.5M chunks with p95 latency under XXX ms, using pgvector HNSW indexes tuned for a measured recall/latency tradeoff.
- Cut LLM cost per query by XX% via a three-layer cache (embedding, semantic, response) with an XX% hit rate.
- Benchmarked 4 chunking strategies and 3 retrieval configurations against a 200-question golden set with RAGAS; hybrid + reranking raised context precision from X to Y.
- FastAPI, pgvector, Redis, Docker, GitHub Actions CI with an automated RAGAS regression gate, langsmith tracing.

Two honest warnings

This is a five-week project at 6+ hours a day. It is genuinely more than v1. If applications start converting, cut scope — the job is the goal.

Cut in this order if you fall behind: langsmith → semantic cache size (drop to 300k chunks; still defensible) → chunking benchmark → corpus.

Never cut: the live URL, the metrics table, CI with the evaluation gate.